

# Project Report3: Secure Private Container Registry with Automated Lifecycle Management

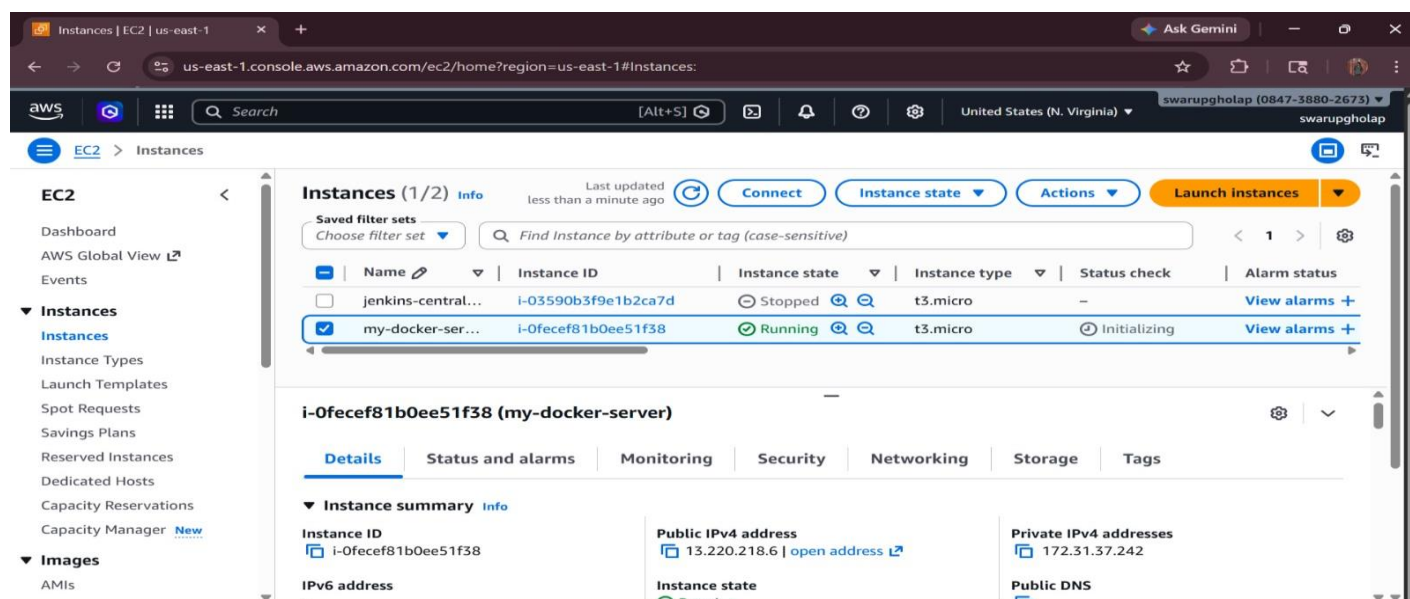
## 1. Project Overview

The goal of this project is to set up a private Amazon ECR (Elastic Container Registry) to securely store Docker images and implement a Lifecycle Policy to automate the deletion of old images, thereby optimizing storage costs.

## 2. Step-by-Step Execution & Screenshots

### Step 1: Provisioning Infrastructure

- Action: Launched an Ubuntu 24.04 LTS EC2 instance named my-docker-server to serve as the build environment.
- Verification: Ensured the instance is in a 'Running' state.



### Step 2: Installing Docker Engine

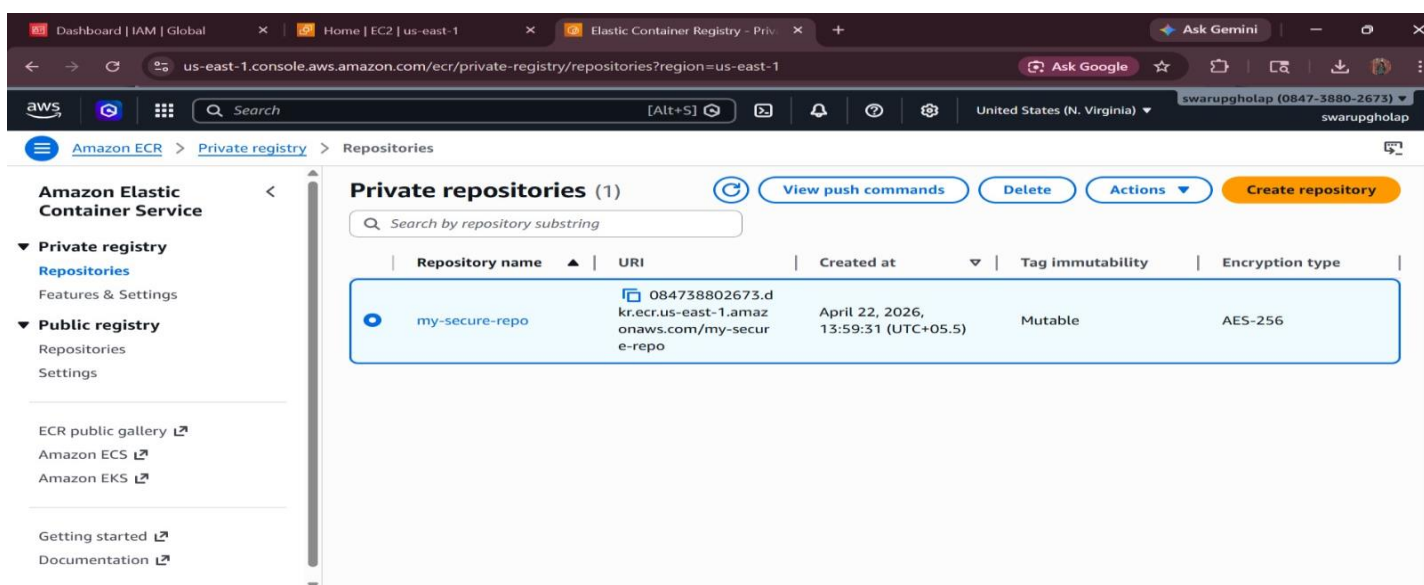
Action: Installed the Docker engine on the Ubuntu instance and configured user permissions to allow Docker commands without sudo.

```
ubuntu@ip-172-31-37-242: ~  
ubuntu @ user manager service: systemd[1020]  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@ip-172-31-37-242:~$ sudo apt install docker.io -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  bridge-utils containerd dns-root-data dnsmasq-base pigz runc ubuntu-fan  
Suggested packages:  
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-doc rinse zfs-fuse | zfsutils  
The following NEW packages will be installed:  
  bridge-utils containerd dns-root-data dnsmasq-base docker.io pigz runc ubuntu-fan  
0 upgraded, 8 newly installed, 0 to remove and 2 not upgraded.  
Need to get 73.8 MB of archives.  
After this operation, 279 MB of additional disk space will be used.  
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]  
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33.9 kB]  
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.3.4-0ubuntu1~24.04.1 [9574 kB]  
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 2.2.1-0ubuntu1~24.04.2 [28.1 MB]  
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 dns-root-data all 2024071801~ubuntu0.24.04.1 [5918 B]  
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/main amd64 dnsmasq-base amd64 2.90-2ubuntu0.1 [376 kB]  
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker.io amd64 29.1.3-0ubuntu3~24.04.1 [35.5 MB]  
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates/universe amd64 ubuntu-fan all 0.12.16+24.04.1 [34.2 kB]  
Fetched 73.8 MB in 1s (81.1 MB/s)  
Preconfiguring packages ...  
Selecting previously unselected package pigz.  
(Reading database ... 104816 files and directories currently installed.)  
Preparing to unpack .../0-pigz_2.8-1_amd64.deb ...  
Unpacking pigz (2.8-1) ...  
Selecting previously unselected package bridge-utils.  
Preparing to unpack .../1-bridge-utils_1.7.1-1ubuntu2_amd64.deb ...  
Unpacking bridge-utils (1.7.1-1ubuntu2) ...  
Selecting previously unselected package runc.  
Preparing to unpack .../2-runc_1.3.4-0ubuntu1~24.04.1_amd64.deb ...
```

```
ubuntu@ip-172-31-37-242: ~  
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.  
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.  
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...  
Processing triggers for man-db (2.12.0-4build2) ...  
Scanning processes...  
Scanning candidates...  
Scanning linux images...  
  
Pending kernel upgrade!  
Running kernel version:  
6.17.0-1007-aws  
Diagnostics:  
The currently running kernel version is not the expected kernel version 6.17.0-1012-aws.  
  
Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.  
  
Restarting services...  
  
Service restarts being deferred:  
/etc/needrestart/restart.d/dbus.service  
systemctl restart networkd-dispatcher.service  
systemctl restart systemd-logind.service  
systemctl restart unattended-upgrades.service  
  
No containers need to be restarted.  
  
User sessions running outdated binaries:  
ubuntu @ session #1: sshd[1014]  
ubuntu @ user manager service: systemd[1020]  
  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@ip-172-31-37-242:~$ sudo systemctl start docker  
ubuntu@ip-172-31-37-242:~$ sudo systemctl enable docker  
ubuntu@ip-172-31-37-242:~$ sudo usermod -aG docker $USER  
ubuntu@ip-172-31-37-242:~$
```

### Step 3: AWS CLI Configuration & ECR Creation

- Action: Configured the AWS CLI with Access Keys and created a private repository named my-secure-repo.
- Command: `aws ecr create-repository --repository-name my-secure-repo`.



### Step 4: Authenticating Docker to ECR

- Action: Used the `get-login-password` command to authenticate the Docker daemon to the private registry.
- Result: Received the "Login Succeeded" confirmation.

```
aws-cli/2.34.34 Python/3.14.4 Linux/6.17.0-1007-aws exe/x86_64.ubuntu.24  
ubuntu@ip-172-31-37-242:~$ aws configure  
  
Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws login'.  
  
AWS Access Key ID [None]: AKIARHOWTQPYX6ODKF75  
AWS Secret Access Key [None]: i0jMSqh8DW1vX9ALLbnH2XIMf00ei7IhAWYtt004  
Default region name [None]: us-east-1  
Default output format [None]: json  
ubuntu@ip-172-31-37-242:~$ aws ecr create-repository --repository-name my-secure-repo --region us-east-1  
{  
  "repository": {  
    "repositoryArn": "arn:aws:ecr:us-east-1:084738802673:repository/my-secure-repo",  
    "registryId": "084738802673",  
    "repositoryName": "my-secure-repo",  
    "repositoryUri": "084738802673.dkr.ecr.us-east-1.amazonaws.com/my-secure-repo",  
    "createdAt": "2026-04-22T08:29:31.108000+00:00",  
    "imageTagMutability": "MUTABLE",  
    "imageScanningConfiguration": {  
      "scanOnPush": false  
    }  
  },  
  "encryptionConfiguration": {  
    "encryptionType": "AES256"  
  }  
}  
  
ubuntu@ip-172-31-37-242:~$ aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 084738802673.dkr.ecr.us-east-1.amazonaws.com/my-secure-repo  
  
WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.  
Configure a credential helper to remove this warning. See  
https://docs.docker.com/go/credential-store/  
  
Login Succeeded  
ubuntu@ip-172-31-37-242:~$ docker build -t my-secure-app:v1 .
```

## Step 5: Image Versioning and Pushing (CI/CD Simulation)

- Action: Built 5 different versions of the container image (v1 to v5) and pushed them to the ECR repository.

```
ubuntu@ip-172-31-37-242: ~/$ cd ecr-project
ubuntu@ip-172-31-37-242: ~/ecr-project$ echo "FROM alpine" > Dockerfile
ubuntu@ip-172-31-37-242: ~/ecr-project$ docker build -t my-secure-app:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/1 : FROM alpine
latest: Pulling from library/alpine
6a0ac1617861: Pulling fs layer
6a0ac1617861: Download complete
a63813f19ccb: Download complete
4e0cb4ac63e7: Download complete
6a0ac1617861: Pull complete
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest
--> 5b10f432ef3d
Successfully built 5b10f432ef3d
Successfully tagged my-secure-app:v1
ubuntu@ip-172-31-37-242: ~/ecr-project$ docker tag my-secure-app:v1 084738802673.dkr.ecr.us-east-1.amazonaws.com/my-secure-repo:v1
ubuntu@ip-172-31-37-242: ~/ecr-project$ docker push 084738802673.dkr.ecr.us-east-1.amazonaws.com/my-secure-repo:v1
The push refers to repository [084738802673.dkr.ecr.us-east-1.amazonaws.com/my-secure-repo]
6a0ac1617861: Pushed
v1: digest: sha256:4d889c14e7d5a73929ab00be2ef8ff22437e7cbc545931e52554a7b00e123d8b size: 1022

Info -> Not all multiplatform-content is present and only the available single-platform image was pushed
sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11 -> sha256:4d889c14e7d5a73929ab00be2ef8ff22437e7cbc54
5931e52554a7b00e123d8b
ubuntu@ip-172-31-37-242: ~/ecr-project$ docker build -t my-secure-app:v2 .
docker tag my-secure-app:v2 084738802673.dkr.ecr.us-east-1.amazonaws.com/my-secure-repo:v2
docker push 084738802673.dkr.ecr.us-east-1.amazonaws.com/my-secure-repo:v2
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/
```

## Step 6: Automating Cleanup with Lifecycle Policy

- Action: Created a lifecycle.json file to define the rule: "If image count is more than 3, expire the oldest images".
- Verification: Applied the policy and checked the "Lifecycle Policy" tab in the console to ensure it is active.

```
GNU nano 7.2                                lifecycle.json
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep only last 3 images",
      "selection": {
        "tagStatus": "any",
        "countType": "imageCountMoreThan",
        "countNumber": 3
      },
      "action": {
        "type": "expire"
      }
    }
  ]
}
```

## 3. Final Proof of Success

- Policy Execution: The PolicyExecutionEvent is visible in the console logs, confirming that AWS has acknowledged the rule and is monitoring the images for cleanup.

Dashboard | IAM | Global

Home | EC2 | us-east-1

Elastic Container Registry | us-east-1

us-east-1.console.aws.amazon.com/ecr/repositories/private/084738802673/my-secure-repo/\_/details?region=us-east-1

swarupgholap (0847-3880-2673)

Amazon ECR > Private registry > Repositories > my-secure-repo

Amazon Elastic Container Service

Private registry

Public registry

my-secure-repo

Lifecycle policy rules

Lifecycle events

Summary

Images

Lifecycle policy

Permissions

Repository tags

With lifecycle policies, you can automatically clean up unused images in your repositories based on rules that you define. When an image meets your policy rules, the action is taken within 24 hours.

|                       | Priority | Description             | Tag status | Tag filters | Match criteria | Action |
|-----------------------|----------|-------------------------|------------|-------------|----------------|--------|
| <input type="radio"/> | 1        | Keep only last 3 images | Any        | -           | Image limit: 3 | Expire |

Lifecycle events

Cloudtrail events from the last 7 days, up to 50 events. To see all events from the past 90 days, use the load all button.

Filter events

| Event time             | Event name           | Message          |
|------------------------|----------------------|------------------|
| Apr 22, 2026, 14:57:55 | PolicyExecutionEvent | 0 images actions |

## 4. Conclusion

The project successfully demonstrates a secure container workflow where images are stored in a private registry and maintained automatically through cloud-native governance policies.